EXPERIMENT 4: SYSY COMPILER CODE GENERATION

# 实验四：SysY 编译器代码生成

从 SysY语言程序代码 到 LLVM IR 生成的完整过程示例

主讲教师：陈安龙

# 实验4内容简介

- 查看指令手册 LLVMRef.mht
- 用 clang 学习 llvm 虚拟指令：

  ① 编写简单的 C 语言程序 test.c

  ② 用 clang -emit-llvm -S test.c -o test.ll

  ③ 生成该文件对应的 llvm 指令
- 学习示例程序，理解代码生成过程

# 实验4内容简介

- 完成算术表达式、逻辑表达式、赋值语句、条件语句、循环语句对应的代码生成，函数命名分别为：
  - ① genArithmeticExpr：生成算术表达式的LLVM IR指令。
  - ② genLogicExpr：生成逻辑表达式的LLVM IR指令。
  - ③ genAssignStmt：生成赋值语句的LLVM IR指令。
  - ④ genIfStmt：生成条件语句的LLVM IR指令。
  - ⑤ genWhileStmt：生成循环语句的LLVM IR指令。
- 函数参数都为：(past node, char* result)
- node 为相关类型的结点
- result 为用来保存LLVM指令，每行只放一条指令；

# 实验安排要求

尽量通过全部测试用例

需完整总结从词法分析到代码生成各步骤的技术要点及相关设计，并体现在实验报告中

提交方式

icoding 平台提交，包括实验4全部代码及实验报告

提交截止日期:

全年级统一，以icoding平台为准

# LLVM IR简介

LLVM IR（Intermediate Representation）是LLVM编译器基础设施中的一种中间表示，旨在为多种编程语言提供一个平台无关的低级表示。LLVM IR具有强大的表达能力，支持多种优化和代码生成。

# LLVM IR特点

① 平台无关性：LLVM IR不依赖于特定的硬件架构，使得编译器可以生成适用于多种平台的代码。

② 静态单赋值（SSA）形式：每个变量在其作用域内只被赋值一次，这使得数据流分析和优化变得更加简单。

③ 强类型系统：LLVM IR具有强类型系统，支持基本数据类型（如整数、浮点数、指针等）和复合数据类型（如结构体、数组等）。

④ 丰富的指令集：LLVM IR提供了丰富的指令集，支持算术运算、逻辑运算、控制流、内存操作等。

# LLVM IR的结构

LLVM IR可以以文本格式或二进制格式表示。文本格式的LLVM IR通常以.ll文件扩展名保存，结构如下：

① 模块（Module）：LLVM IR的顶层结构，包含全局变量、函数和其他定义。

② 函数（Function）：模块中的每个函数都有一个名称、返回类型和参数类型。

③ 基本块（Basic Block）：函数由多个基本块组成，每个基本块是一个顺序执行的代码序列，包含控制流指令。

④ 指令（Instruction）：基本块中的每条指令执行特定的操作，如算术运算、内存访问、控制流等。

# LLVM IR的基本语法

- 注释：以;开头的行是注释。
- 类型：LLVM IR支持多种数据类型，如i32（32位整数）、float（浮点数）、double（双精度浮点数）、i8*（指向8位整数的指针）等。
- 指令格式：指令通常以操作符开头，后跟操作数。例如：

```
%result = add i32 %a, %b
```

# 算术指令

- 加法：add指令用于两个操作数的加法。

```
%result = add i32 %a, %b
```

- 减法：sub指令用于两个操作数的减法。

```
%result = sub i32 %a, %b
```

- sdiv指令用于执行有符号整数除法；udiv指令用于执行无符号整数除法。

```
%result_sdiv = sdiv i32 %a, %b  ；计算 %a / %b（有符号除法）

%result_udiv = udiv i32 %c, %d  ；计算 %c / %d（无符号除法）
```

# 浮点运算指令

- 浮点加法：fadd指令用于执行浮点加法。

```
%result = fadd float %a, %b  ; 计算 %a + %b
```

- 浮点减法：fsub指令用于执行浮点减法。。

```
%result = fsub float %a, %b  ; 计算 %a - %b
```

- fdiv指令用于执行浮点除法；fmul指令用于执行浮点乘法。

```
%result = fdiv float %a, %b  ; 计算 %a / %b

%result = fmul float %a, %b  ; 计算 %a * %b
```

# 逻辑指令：

- 与运算：and指令用于按位与运算。

```
%result = and i32 %a, %b
```

- 或运算：or指令用于按位或运算。

```
%result = or i32 %a, %b
```

- not指令用于执行按位非运算。

```
%result_not = not i1 %condition  ; 计算 %condition 的非
```

# 比较运算：

- 整数比较：icmp指令用于整数比较。

```
%result_eq = icmp eq  i32 %a, %b  ; 等于比较

%result_gt = icmp gt i32 %a, %b  ; 大于比较

%result_ge = icmp ge i32 %a, %b  ; 大于等于比较

%result_lt = icmp lt i32 %a, %b   ; 小于比较

%result_le = icmp le i32 %a, %b  ; 小于等于比较
```

# 浮点比较运算：

- fcmp指令用于比较两个浮点数。
- 语法：%result = fcmp <predicate> <type> <operand1>, <operand2>
- <predicate>：比较条件，如oeq（等于）、ogt（大于）、oge（大于等于）、olt（小于）、ole（小于等于）等。
- 示例：

① %result = fcmp oeq float %a, %b

; 如果 %a 等于 %b，则 %result_eq 为 true (1)，否则为 false (0)

② %result = fcmp ogt float %a, %b

; 如果 %a 大于 %b，则 %result_gt 为 true (1)，否则为 false (0)

# 控制流指令：

- 条件跳转：br指令用于条件跳转。

  ```
  br i1 %condition, label %true_block, label %false_block
  ```

- 无条件跳转：br指令用于无条件跳转。

  ```
  br label %next_block
  ```

# 内存操作指令：

- 加载：load指令用于从内存中加载值。

① 语法：%value = load <type>, <type>* <pointer>

② 示例：%loaded_value = load i32, i32* %ptr ；从 %ptr 指向的内存位置加载值

- 存储：store指令用于将值存储到内存中。

① 语法：store <type> <value>, <type>* <pointer>

② 示例：store i32 %value, i32* %ptr ；将值存储到 %ptr 指向的内存位置

# 内存分配指令：

- alloca指令用于在栈上分配内存。它的语法如下：

① 语法：%ptr = alloca <type>, <alignment>

<type>：要分配的类型。

<alignment>（可选）：内存对齐。

② 示例：%ptr = alloca i32 align 4; 在栈上分配一个32位整数

# 函数声明：

- 函数声明用于定义函数的签名，包括返回类型、函数名称和参数类型。函数声明不会包含函数的实现。

① 语法：declare <return_type> @<function_name>(<parameter_types>)

<return_type>：函数的返回类型。

@<function_name>：函数的名称，前面带有@符号。

<parameter_types>：函数参数的类型列表，多个参数用逗号分隔。

② 示例：declare i32 @add(i32, i32); 简单的函数声明，声明一个返回i32类型并接受两个i32类型参数的函数。

# 函数定义：

- 函数定义包含函数的实现，包括函数体和指令。

① 语法：define <return_type> @<function_name>(<parameter_types>) {

<function_body>

}

② 示例：简单的函数定义，定义一个返回两个整数和的函数：

```
define i32 @add(i32 %a, i32 %b) {
entry:
    %sum = add i32 %a, %b
    ret i32 %sum
}
```

# 函数调用：

- 函数调用使用call指令。调用指令用于执行已声明或定义的函数，并获取返回值。

① 语法：%result = call <return_type> @<function_name>(<arguments>)

%result：存储函数返回值的变量。

<return_type>：函数的返回类型。

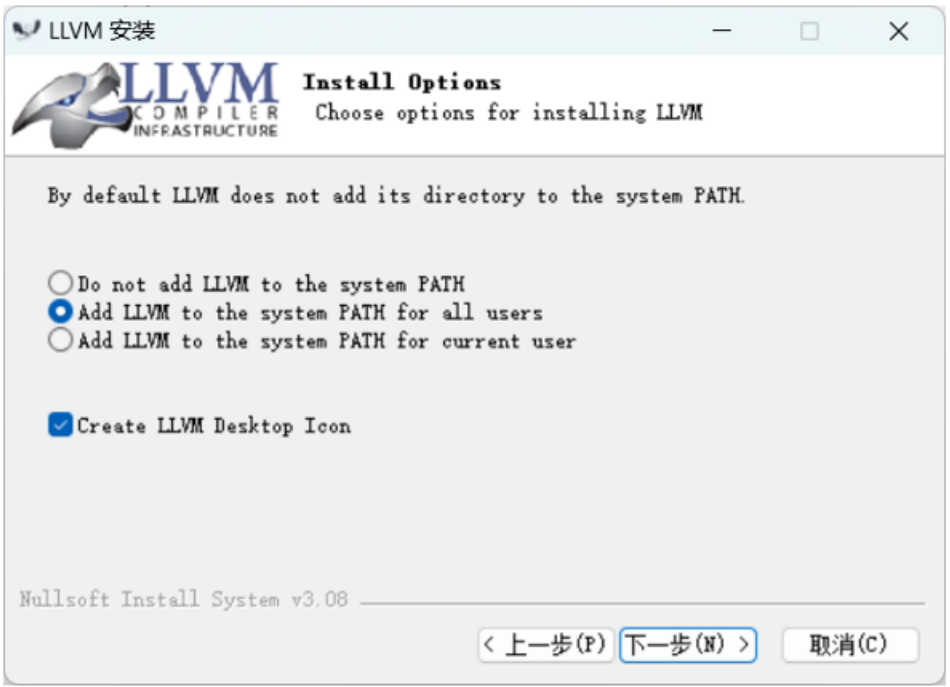
@<function_name>：要调用的函数名称。

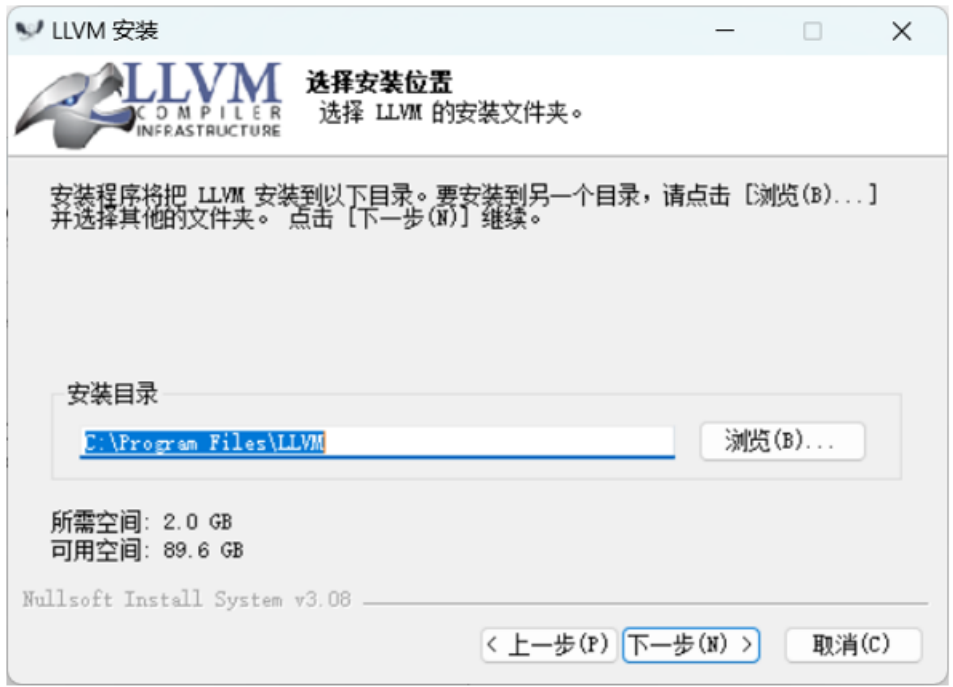
<arguments>：传递给函数的参数列表。}

② 示例：调用之前定义的add函数的示例：

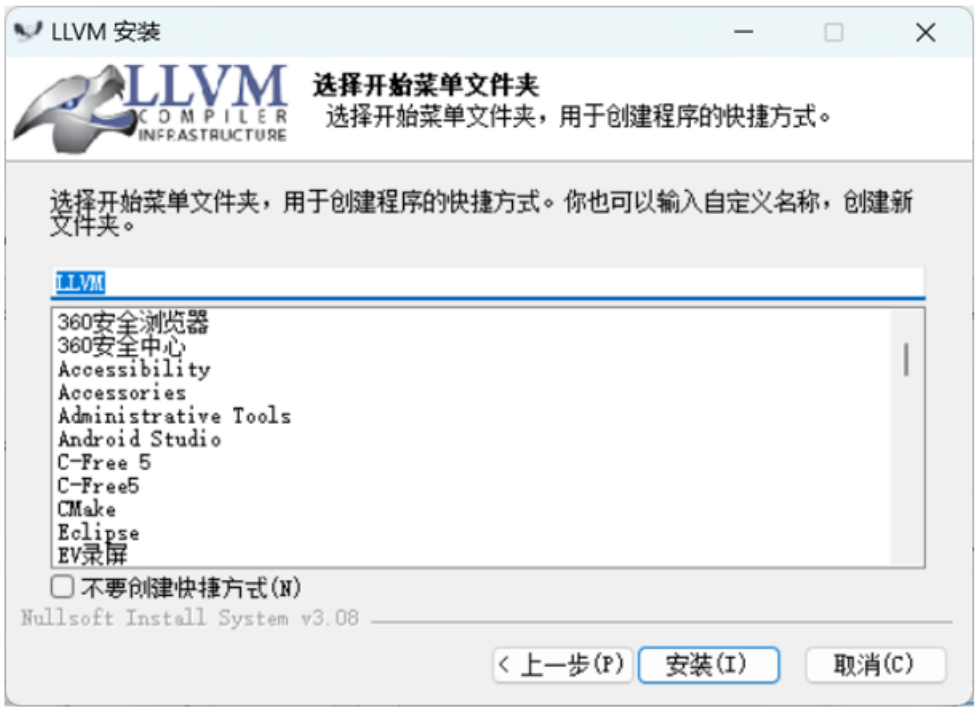
```
%result = call i32 @add(i32 5, i32 10) ；调用 add 函数，传递 5 和 10
```

# 安装LLVM


LLVM 安装
Install Options
Choose options for installing LLVM
By default LLVM does not add its directory to the system PATH.
Do not add LLVM to the system PATH
Add LLVM to the system PATH for all users
Add LLVM to the system PATH for current user
Create LLVM Desktop Icon
Nullsoft Install System v3.08
< 上一步(P)
下一步(N) >
取消(C)


LLVM 安装
选择安装位置
选择 LLVM 的安装文件夹。
安装程序将把 LLVM 安装到以下目录。要安装到另一个目录，请点击 [浏览(B)...] 并选择其他的文件夹。 点击 [下一步(N)] 继续。
安装目录
C:\Program Files\LLVM
浏览(B)...
所需空间: 2.0 GB
可用空间: 89.6 GB
Nullsoft Install System v3.08
< 上一步(P)
下一步(N) >
取消(C)

# 安装LLVM


LLVM 安装
选择开始菜单文件夹
选择开始菜单文件夹，用于创建程序的快捷方式。
选择开始菜单文件夹，用于创建程序的快捷方式。你也可以输入自定义名称，创建新文件夹。
LLVM
360安全浏览器
360安全中心
Accessibility
Accessories
Administrative Tools
Android Studio
C-Free 5
C-Free5
CMake
Eclipse
EV录屏
不要创建快捷方式(N)
Nullsoft Install System v3.08
< 上一步(P)
安装(I)
取消(C)

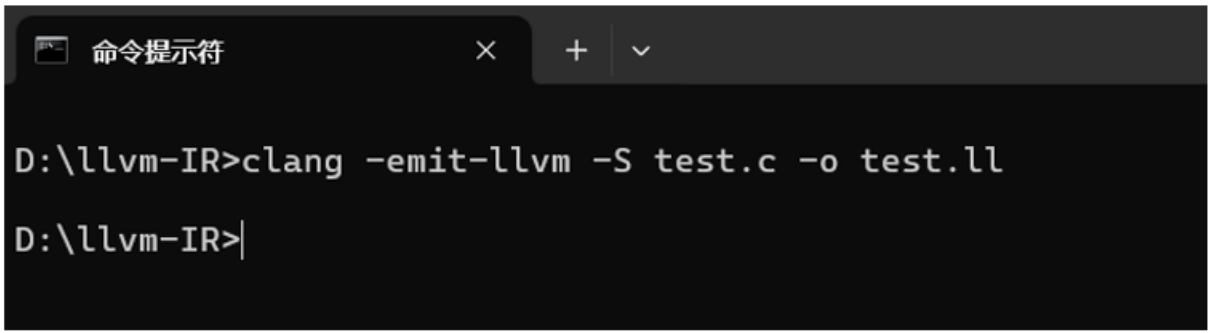

LLVM 安装
正在安装
LLVM 正在安装，请稍候。
解压缩: clang++.exe... 38%
显示详情(D)
Nullsoft Install System v3.08
< 上一步(P)
下一步(N) >
取消(C)

# 假如编写test.c程序

```c
int mytest()
{
    int a = 5;
    int b = 3;
    int c;
    if (a > b)
    {
        c = a * b;
        a = b + 2;
    }
    else
        c = a + b;
    while (a <= c)
    {
        a = a + b;
        b = b - 1;
    }
    return c;
}
```

# clang 生成中间代码

```
clang -emit-llvm -S test.c -o test.ll
```



# test.c的中间代码如下：

test.ll

```
; ModuleID = 'test.c'
source_filename = "test.c"
target datalayout = "e-m:w-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-f80:128-n8:16:32:64-S128"
target triple = "x86_64-pc-windows-msvc19.42.34321"

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @mytest() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  store i32 5, ptr %1, align 4
  store i32 3, ptr %2, align 4
  %4 = load i32, ptr %1, align 4
  %5 = load i32, ptr %2, align 4
  %6 = icmp sgt i32 %4, %5
  br i1 %6, label %7, label %13

```

```c
int mytest()
{
    int a = 5;
    int b = 3;
    int c;
    if (a > b)
    {
        c = a * b;
        a = b + 2;
    }
    else
        c = a + b;
    while (a <= c)
    {
        a = a + b;
        b = b - 1;
    }
    return c;
}
```

# test.c的中间代码如下：

```
7:                                                ; preds = %0
  %8 = load i32, ptr %1, align 4
  %9 = load i32, ptr %2, align 4
  %10 = mul nsw i32 %8, %9
  store i32 %10, ptr %3, align 4
  %11 = load i32, ptr %2, align 4
  %12 = add nsw i32 %11, 2
  store i32 %12, ptr %1, align 4
  br label %17

13:                                               ; preds = %0
  %14 = load i32, ptr %1, align 4
  %15 = load i32, ptr %2, align 4
  %16 = add nsw i32 %14, %15
  store i32 %16, ptr %3, align 4
  br label %17

17:                                               ; preds = %13, %7
  br label %18
```

```
int mytest()
{
    int a = 5;
    int b = 3;
    int c;
    if (a > b)
    {
        c = a * b;
        a = b + 2;
    }
    else
        c = a + b;
    while (a <= c)
    {
        a = a + b;
        b = b - 1;
    }
    return c;
}
```

# test.c的中间代码如下：

```
34
35  17:                                               ; preds = %13, %7
36    br label %18
37
38  18:                                               ; preds = %22, %17
39    %19 = load i32, ptr %1, align 4
40    %20 = load i32, ptr %3, align 4
41    %21 = icmp sle i32 %19, %20
42    br i1 %21, label %22, label %28
43
44  22:                                               ; preds = %18
45    %23 = load i32, ptr %1, align 4
46    %24 = load i32, ptr %2, align 4
47    %25 = add nsw i32 %23, %24
48    store i32 %25, ptr %1, align 4
49    %26 = load i32, ptr %2, align 4
50    %27 = sub nsw i32 %26, 1
51    store i32 %27, ptr %2, align 4
52    br label %18, !llvm.loop !5
53
54  28:                                               ; preds = %18
55    %29 = load i32, ptr %3, align 4
56    ret i32 %29
57  }
58
```

```
int mytest()
{
    int a = 5;
    int b = 3;
    int c;
    if (a > b)
    {
        c = a * b;
        a = b + 2;
    }
    else
        c = a + b;
    while (a <= c)
    {
        a = a + b;
        b = b - 1;
    }
    return c;
}
```

# 代码生成示例：字符流到 LLVM IR 文本的单向四阶段

main 驱动 parse → semantic_analyze → ir_gen_module，依赖严格单向、无环




**ast.c** 不参与流水线调度，只提供 AST 结点构造（make* 系列）与释放（freeAst），被 parser 与 ir_gen 广泛复用。

**gencode.c**（main）是唯一驱动者，串联四个阶段并负责读文件、分配 IR 缓冲区、打印结果。

ir_gen.c 内部再通过 genStmt / genExpr / genArithmeticExpr / genLogicExpr / genAssignStmt 五者互相递归，完成各类语句与表达式的翻译。

# 六个模块的职责与对外入口

每个模块只暴露一个或少数几个外部入口，其余函数均为 static 内部辅助

| 模块 | 对外入口 | 职责 | 函数数（外部/static） |
| --- | --- | --- | --- |
| gencode.c | `main()` | 读源码、驱动流水线、打印 IR、释放 AST | 1 / 0 |
| lexer.c | `lexer_init / yylex / token_name` | 手写词法分析器，输出 token 与 yylval | 3 / 5 |
| parser.c | `parse(src)` | 递归下降语法分析，分层构造 AST | 1 / 15 |
| ast.c | `make* / appendSibling / freeAst` | AST 结点构造与释放工具，纯 libc | 10 / 0 |
| semantic.c | `semantic_init / analyze / lookup_sym …` | 作用域栈、符号登记、类型回填 | 7 / 1 |
| ir_gen.c | `ir_gen_module` + 五大核心函数 | 遍历 AST，逐结点生成 LLVM IR 指令 | 10 / 8 |

# main 入口：四步驱动整条流水线

gencode.c 唯一的 main() 调用其它模块的全部外部入口

```
main(argc, argv)
├─ 读源码（fopen / fread / getchar）
│
├─ parse(src) // parser.c
│  └─ lexer_init + parseCompUnit → AST
│
├─ semantic_init() // semantic.c
├─ semantic_analyze(root) // semantic.c
│  └─ declare_sym / enter_scope / expr_type
│
├─ ir_gen_init() // ir_gen.c
├─ ir_gen_module(root, ir, 1MB)
│  └─ genStmt / genExpr / genArithmeticExpr …
│
├─ printf("%s", ir)
├─ freeAst(root) // ast.c
└─ free(src) / free(ir)
```

## 关键观察

**1. 唯一驱动者**

main 是唯一的调度入口，其它模块之间互不感知。

**2. 缓冲区一次分配**

ir 输出缓冲固定 1 MB，ir_gen_module 直接写入。

**3. AST 一次释放**

freeAst 递归释放 left / right / next 三条链。

**4. 错误恢复弱**

fopen 失败直接 return 1，语法错误只打 stderr。

# 词法 + 语法：yylex 被 advance 反复驱动

parser 是一棵自顶向下递归下降树，每层只调用下一层表达式构造二元结点

## lexer.c 内部

**yylex()**

```
├─ skip_ws() → peek()
├─ read_int() → peek / next_char
├─ read_id() → peek / next_char
└─ keyword()
```

其它对外：

```
├─ lexer_init(src)
└─ token_name(t) // 报错用
```

## parser.c 表达式分层

```
parse()
└─ parseCompUnit()
├─ parseDecl / parseBlock
└─ parseStmt → parseLOrExp

parseLOrExp   ┐
parseLAndExp  ├ 循环调下一层
parseEqExp    │ 遇算符 makeBinaryNode
parseRelExp   │
parseAddExp   │
parseMulExp   │
parseUnaryExp ┘
└─ parsePrimaryExp（终结）
```

# 语义分析：作用域栈 + 类型递归推导

ast.c 只构造与释放，不参与调度；semantic.c 沿 AST 兄弟链逐结点登记符号

### ast.c 工具函数（被 parser / ir_gen 调用）

```
makeIntNode / makeFloatNode
makeIdNode / makeUnaryNode
makeBinaryNode / makeNode
makeIfNode / makeWhileNode
appendSibling(head, node)

freeAst(root) 递归三条链：
├─ freeAst(left)
├─ freeAst(right)
└─ freeAst(next)
```

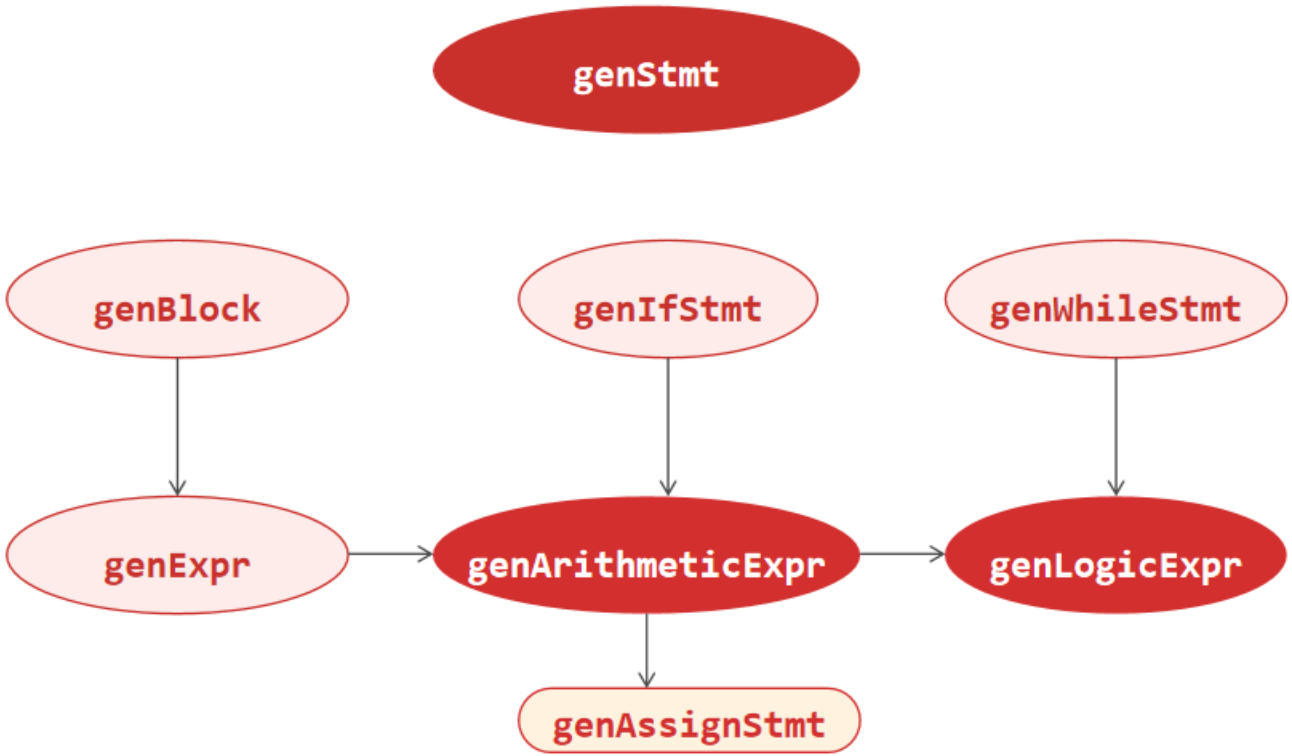
### semantic.c 内部调用

```
semantic_analyze(root)
├─ declare_sym (全局变量 / 函数)
├─ enter_scope
│  └─ declare_sym (局部变量)
├─ expr_type(node) static
│  ├─ lookup_sym (DECL_REF_EXPR)
│  └─ expr_type(left / right) 递归
└─ leave_scope

对外：lookup_sym 被 ir_gen 复用
```

# IR 生成：五个核心函数互相递归

ir_gen_module 是入口，genStmt / genExpr / genArithmeticExpr / genLogicExpr / genAssignStmt 五者互调



# 从SYSY语言到代码生成示例

```
/* token头文件 */
#ifndef COMPILER_LAB_TOKEN_H
#define COMPILER_LAB_TOKEN_H
enum yytokentype {
    num_INT = 258,
    num_FLOAT = 259,

    Y_ID = 260,
    Y_INT = 261,
    Y_VOID = 262,
    Y_CONST = 263,

    Y_IF = 264,
    Y_ELSE = 265,
    Y_WHILE = 266,
    Y_BREAK = 267,
    Y_CONTINUE = 268,
    Y_RETURN = 269,
    Y_ADD = 270,
    Y_SUB = 271,
    Y_MUL = 272,
    Y_DIV = 273,
    Y_MODULO = 274,

    Y_LESS = 275,
    Y_LESSEQ = 276,
    Y_GREAT = 277,
    Y_GREATEQ = 278,
    Y_NOTEQ = 279,
    Y_EQ = 280,
    Y_NOT = 281,
    Y_AND = 282,
    Y_OR = 283,
    Y_ASSIGN = 284,
    Y_LPAR = 285,
    Y_RPAR = 286,
    Y_LBRACKET = 287,
    Y_RBRACKET = 288,
    Y_LSQUARE = 289,
    Y_RSQUARE = 290,
    Y_COMMA = 291,
    Y_SEMICOLON = 292,
    Y_FLOAT = 293
};
typedef union_YYLVAL{
    int     token;
    int     int_value;
    float   float_value;
    char*   id_name;
}_YYLVAL;
extern _YYLVAL yylval;
#endif //COMPILER_LAB_TOKEN_H
```

# 定义词法分析器的头文件lexer.h

```
#ifndef COMPILER_LEXER_H
#define COMPILER_LEXER_H
#include "token.h"
#include <stdio.h>
extern _YYLVAL yylval;
void  lexer_init(const char* src);
int   yylex(void);
constchar* token_name(int t);
#endif
```

# 词法分析器lexer.c文件

```c
#include"lexer.h"
#include<ctype.h>
#include<string.h>
#include<stdlib.h>

_YYLVAL yylval;

static const char* g_src = NULL;
static int g_pos = 0;

void lexer_init(const char* src) {
    g_src = src;
    g_pos = 0;
}
static int peek(void) { return g_src[g_pos]; }
static int next_char(void) { return g_src[g_pos++]; }
```

# 词法分析器lexer.c文件（续）

```c
/* 跳过空白和注释 */
static void skip_ws(void) {
    for (;;) {
        while (isspace((unsigned char)peek())) g_pos++;
        if (peek() == '/' && g_src[g_pos+1] == '/') {
            while (peek() != '\n' && peek() != '\0') g_pos++;
        } elseif (peek() == '/' && g_src[g_pos+1] == '*') {
            g_pos += 2;
            while (!(peek() == '*' && g_src[g_pos+1] == '/') && peek() != '\0')
             g_pos++;
            if (peek() != '\0') g_pos += 2;
        } else break;
    }
}
```

# 词法分析器lexer.c文件（续）

```c
static int read_int(void) {
    int v = 0;
    while (isdigit((unsignedchar)peek())) v = v*10 + (next_char() - '0');
    return v;
}
static int read_id(char* out) {
    int n = 0;
    while (isalnum((unsignedchar)peek()) || peek() == '_') out[n++] = next_char();
    out[n] = '\0';
    return n;
}
```

# 词法分析器lexer.c文件（续）

```c
static int keyword(const char* s) {
    if (strcmp(s, "int")    == 0) return Y_INT;
    if (strcmp(s, "void")   == 0) return Y_VOID;
    if (strcmp(s, "const")  == 0) return Y_CONST;
    if (strcmp(s, "if")     == 0) return Y_IF;
    if (strcmp(s, "else")   == 0) return Y_ELSE;
    if (strcmp(s, "while")  == 0) return Y_WHILE;
    if (strcmp(s, "break")    == 0) return Y_BREAK;
    if (strcmp(s, "continue") == 0) return Y_CONTINUE;
    if (strcmp(s, "return")   == 0) return Y_RETURN;
    if (strcmp(s, "float")  == 0) return Y_FLOAT;
    return 0;
}
```

# 词法分析器lexer.c文件（续）

```c
int yylex(void) {
    skip_ws();
    int c = peek();
    if (c == '\0') return 0;
    if (isdigit((unsigned char)c)) {
        int v = read_int();
        if (peek() == '.') {
            g_pos--; /* 回退刚读到的数字起点 */
            /* 简单处理：重新读带小数 ，为简化，先回退到数字起点附近 */
            /* 更稳妥做法：用 read_float 从头读 ，这里直接判断后续是否为点 */
            /* 上面 read_int 已消耗完整数部分，peek 是 '.' */
            next_char(); /* consume '.' */
            while (isdigit((unsignedchar)peek())) next_char();
            /* 还原：把已消费的数字存回 */
            /* 为简化，直接设置 float_value */
            /* 用 sscanf 不现实，因为已消耗。这里近似：把 v 转成 float */
            yylval.float_value = (float)v;
            return num_FLOAT;
        }
        yylval.int_value = v;
        return num_INT;
    }
```

# 词法分析器lexer.c文件（续）

```
    if (isalpha((unsignedchar)c) || c == '_') {
        char buf[256];
        read_id(buf);
        int kw = keyword(buf);
        if (kw) return kw;
        yylval.id_name = strdup(buf);
        return Y_ID;
     }

    /* 双字符运算符 */
    if (c == '=' && g_src[g_pos+1] == '=') { g_pos += 2; return Y_EQ; }
    if (c == '!' && g_src[g_pos+1] == '=') { g_pos += 2; return Y_NOTEQ; }
    if (c == '<'  && g_src[g_pos+1] == '=') { g_pos += 2; return Y_LESSEQ; }
    if (c == '>'  && g_src[g_pos+1] == '=') { g_pos += 2; return Y_GREATEQ; }
    if (c == '&'  && g_src[g_pos+1] == '&') { g_pos += 2; return Y_AND; }
    if (c == '|'  && g_src[g_pos+1] == '|') { g_pos += 2; return Y_OR; }
```

# 词法分析器lexer.c文件（续）

```
g_pos++;
    switch (c) {
        case'+': return Y_ADD;
        case'-': return Y_SUB;
        case'*': return Y_MUL;
        case'/': return Y_DIV;
        case'%': return Y_MODULO;
        case'<': return Y_LESS;
        case'>': return Y_GREAT;
        case'!': return Y_NOT;
        case'=': return Y_ASSIGN;
        case'(': return Y_LPAR;
        case')': return Y_RPAR;
        case'{': return Y_LBRACKET;
        case'}': return Y_RBRACKET;
        case'[': return Y_LSQUARE;
        case']': return Y_RSQUARE;
        case',': return Y_COMMA;
        case';': return Y_SEMICOLON;
        default: return 0;
    }
}
```

# 词法分析器lexer.c文件（续）

```
const char* token_name(int t) {
    switch (t) {
        case num_INT: return "num_INT";
        case num_FLOAT: return "num_FLOAT";
        case Y_ID: return "ID";
        case Y_INT: return "int";
        case Y_VOID: return "void";
        case Y_CONST: return "const";
        case Y_IF: return "if";
        case Y_ELSE: return "else";
        case Y_WHILE: return "while";
        case Y_BREAK: return "break";
        case Y_CONTINUE: return "continue";
        case Y_RETURN: return "return";
        case Y_ADD: return "+";
        case Y_SUB: return "-";
        case Y_MUL: return "*";
        case Y_DIV: return "/";
        case Y_MODULO: return "%";
        caseY_LESS: return "<";
        case Y_LESSEQ: return "<=";
        case Y_GREAT: return ">";
        case Y_GREATEQ: return ">=";
        case Y_NOTEQ: return "!=";
        case Y_EQ: return "==";
        case Y_NOT: return "!";
        case Y_AND: return "&&";
        case Y_OR: return "||";
        case Y_ASSIGN: return "=";
        case Y_LPAR: return "(";
        case Y_RPAR: return ")";
        case Y_LBRACKET: return "{";
        case Y_RBRACKET: return "}";
        case Y_LSQUARE: return "[";
        case Y_RSQUARE: return "]";
        case Y_COMMA: return ",";
        case Y_SEMICOLON: return ";";
        case Y_FLOAT: return "float";
        default: return "?";
    }
}
```

# node_type.h头文件

```
#ifndef COMPILER_LAB_NODE_TYPE_H
#define COMPILER_LAB_NODE_TYPE_H
enum _node_type{
    STRUCT_DECL = 2,
    UNION_DECL = 3,
    ENUM_DECL = 5,
    FIELD_DECL = 6,
    ENUM_CONSTANT_DECL = 7,
    FUNCTION_DECL = 8,
    VAR_DECL = 9,
    PARM_DECL = 10,
    TYPEDEF_DECL = 20,
    TYPE_ALIAS_DECL = 36,
    MEMBER_REF = 47,
    LABEL_REF = 48,
    OVERLOADED_DECL_REF = 49,
    VARIABLE_REF = 50,
    INVALID_FILE = 70,
    NO_DECL_FOUND = 71,
    NOT_IMPLEMENTED = 72,
    INVALID_CODE = 73,
    UNEXPOSED_EXPR = 100,
    DECL_REF_EXPR = 101,
    MEMBER_REF_EXPR = 102,
    CALL_EXPR = 103,
    BLOCK_EXPR = 105,
    INTEGER_LITERAL = 106,
    FLOATING_LITERAL = 107,
    IMAGINARY_LITERAL = 108,
    STRING_LITERAL = 109,
    CHARACTER_LITERAL = 110,
    PAREN_EXPR = 111,
    UNARY_OPERATOR = 112,
    ARRAY_SUBSCRIPT_EXPR = 113,
```

# node_type.h头文件

```
    COMPOUND_ASSIGNMENT_OPERATOR = 115,
    CONDITIONAL_OPERATOR = 116,
    CSTYLE_CAST_EXPR = 117,
    COMPOUND_LITERAL_EXPR = 118,
    INIT_LIST_EXPR = 119,
    ADDR_LABEL_EXPR = 120,
    UNEXPOSED_STMT = 200,
    LABEL_STMT = 201,
    COMPOUND_STMT = 202,
    CASE_STMT = 203,
    DEFAULT_STMT = 204,
    IF_STMT = 205,
    SWITCH_STMT = 206,
    WHILE_STMT = 207,
    DO_STMT = 208,
    FOR_STMT = 209,
    GOTO_STMT = 210,
    INDIRECT_GOTO_STMT = 211,
    CONTINUE_STMT = 212,
    BREAK_STMT = 213,
    RETURN_STMT = 214,
    NULL_STMT = 230,
    DECL_STMT = 231,
    TRANSLATION_UNIT = 300,
};
typedef enum _node_type node_type;
#endif
```

# 抽象语法树ast.h头文件

```c
#ifndef COMPILER_AST_H
#define COMPILER_AST_H
#include "node_type.h"
#include "token.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
/* 抽象语法树结点定义 */
typedef struct _ast ast;
typedef struct _ast *past;
```

# 抽象语法树ast.h头文件(续)

```
struct _ast {
    int          ivalue;     /* 整数字面量 / 运算符 token / 类型编码 */
    float        fvalue;     /* 浮点字面量 */
    char*        svalue;     /* 标识符名 / 函数名 */
    node_type    nodeType;   /* 结点类型 */
    past         left;       /* 左孩子（第一个孩子） */
    past         right;      /* 右孩子（第二个孩子） */
    past         if_cond;    /* if 语句的条件表达式（while 也复用此字段存条件） */
    past         next;       /* 兄弟链（同层并列结点） */
    /* ---- 以下为语义/代码生成附加字段 ---- */
    int          dataType;   /* 结点求值结果类型: 0=void, 1=int, 2=float */
    char*        irName;     /* 该结点在 LLVM IR 中的结果寄存器名（如 %t3） */
    int          isLValue;   /* 是否为左值 */
};
```

# 抽象语法树ast.h头文件(续)

```
/* 创建叶子/一元/二元结点的工具函数 */
past makeIntNode(int val);
past makeFloatNode(float val);
past makeIdNode(node_type t, const char* name);
past makeUnaryNode(node_type t, int op, past child);
past makeBinaryNode(node_type t, int op, past l, past r);
past makeNode(node_type t, past left, past right);
past makeIfNode(past cond, past thenStmt, past elseStmt);
past makeWhileNode(past cond, past bodyStmt);
past appendSibling(past head, past node);
/* 类型编码常量 */
#define TY_VOID 0
#define TY_INT  1
#define TY_FLOAT 2
/* 释放整棵树 */
void freeAst(past root);
#endif
```

# 抽象语法树ast.c文件

```c
#include "ast.h"
past makeIntNode(int val) {
    past p = (past)calloc(1, sizeof(ast));
    p->nodeType = INTEGER_LITERAL;
    p->ivalue = val;
    p->dataType = TY_INT;
    return p;
}
past makeFloatNode(float val) {
    past p = (past)calloc(1, sizeof(ast));
    p->nodeType = FLOATING_LITERAL;
    p->fvalue = val;
    p->dataType = TY_FLOAT;
    return p;
}
```

# 抽象语法树ast.c文件（续）

```
past makeIdNode(node_type t, constchar* name) {
    pastp = (past)calloc(1, sizeof(ast));
    p->nodeType = t;
    p->svalue = strdup(name);
    return p;
}
past makeUnaryNode(node_type t, int op, past child) {
    past p = (past)calloc(1, sizeof(ast));
    p->nodeType = t;
    p->ivalue = op;
    p->left = child;
    return p;
}
```

# 抽象语法树ast.c文件（续）

```
past makeBinaryNode(node_type t, int op, past l, past r) {
    pastp = (past)calloc(1, sizeof(ast));
    p->nodeType = t;
    p->ivalue = op;
    p->left = l;
    p->right = r;
    return p;
}

past makeNode(node_type t, past left, past right) {
    pastp = (past)calloc(1, sizeof(ast));
    p->nodeType = t;
    p->left = left;
    p->right = right;
    return p;
}
```

# 抽象语法树ast.c文件（续）

```
past makeIfNode(past cond, past thenStmt, past elseStmt) {
    past p = (past)calloc(1, sizeof(ast));
    p->nodeType = IF_STMT;
    p->if_cond = cond;
    p->left = thenStmt;
    p->right = elseStmt;   /* else 可能为 NULL */
    return p;
}
past makeWhileNode(past cond, past bodyStmt) {
    past p = (past)calloc(1, sizeof(ast));
    p->nodeType = WHILE_STMT;
    p->if_cond = cond;
    p->left = bodyStmt;
    return p;
}
```

# 抽象语法树ast.c文件（续）

```
past appendSibling(past head, past node) {
    if (!head) return node;
    past cur = head;
    while (cur->next) cur = cur->next;
    cur->next = node;
    return head;
}
void freeAst(past root) {
    if (!root) return;
    freeAst(root->left);
    freeAst(root->right);
    freeAst(root->next);
    if (root->svalue) free(root->svalue);
    if (root->irName) free(root->irName);
    free(root);
}
```

# 语法分析器parser.h头文件

```
#ifndef COMPILER_PARSER_H
#define COMPILER_PARSER_H
#include "ast.h"
/* 对源字符串做递归下降语法分析，返回 AST 根 */
past parse(const char* src);
#endif
```

# 语法分析器parser.c文件

```c
#include "parser.h"
#include "lexer.h"
#include <stdio.h>
#include <string.h>
static int  g_look;
static int  g_type;     /* 当前 Type (Y_INT/Y_FLOAT/Y_VOID) */
static void advance(void)  { g_look = yylex(); }
static void match(int t) {
    if (g_look == t) advance();
    else {
        fprintf(stderr, "syntax error: expected %s, got %s\n",
                token_name(t), token_name(g_look));
    }
}
```

# 语法分析器parser.c文件

```c
/* 前向声明 */
static past parseCompUnit(void);
static past parseDecl(void);
static past parseBlock(void);
static past parseStmt(void);
static past parseExp(void);
static past parseAddExp(void);
static past parseMulExp(void);
static past parseUnaryExp(void);
static past parsePrimaryExp(void);
static past parseRelExp(void);
static past parseEqExp(void);
static past parseLAndExp(void);
static past parseLOrExp(void);
```

# 语法分析器parser.c文件

```
/* Type: int | float | void */
static int parseType(void) {
    int t = g_look;
    if (t == Y_INT || t == Y_FLOAT || t == Y_VOID) { advance(); return t; }
    return 0;
}
```

# 语法分析器parser.c文件

```
/* CompUnit: Decl | FuncDef | CompUnit */
static past parseCompUnit(void) {
    past head = NULL;
    while (g_look != 0) {
        past item = NULL;
        if (g_look == Y_CONST) {
            item = parseDecl();
        } else if (g_look == Y_INT || g_look == Y_FLOAT || g_look == Y_VOID) {
            /* 可能是变量声明或函数定义：看后面是否跟 ( */
            g_type = parseType();
            if (g_look == Y_ID) {
                /* 看 Y_ID 后面，这里简化：直接根据下一个 token 判断 */
                /* 为了不回退，用试探法,简化：如果下下个是 ( 则函数，否则变量 */
                /* 实际写法：先把 ID 存下来，match 后看 */
                char* name = yylval.id_name;
                advance(); /* 吃 ID */
```

# 语法分析器parser.c文件

```
if (g_look == Y_LPAR) {
    /* 函数定义 */
    past fn = makeNode(FUNCTION_DECL, NULL, NULL);
    fn->svalue = strdup(name);
    fn->ivalue = g_type;
    match(Y_LPAR);
    /* 简化：忽略参数列表 */
    while (g_look != Y_RPAR && g_look != 0) advance();
    match(Y_RPAR);
    past body = parseBlock();
    fn->left = body;
    item = fn;
} else {
    /* 变量声明：int a = ...; 简化为单个变量 */
    past vd = makeNode(VAR_DECL, NULL, NULL);
    vd->svalue = strdup(name);
    vd->ivalue = g_type;
```

# 语法分析器parser.c文件

```
                    if (g_look == Y_ASSIGN) {
                        advance();
                        vd->right = parseExp();
                    }
                    match(Y_SEMICOLON);
                    item = vd;
                }
            }
        } else {
            advance(); /* 容错 */
        }
        head = appendSibling(head, item);
    }
    past root = makeNode(TRANSLATION_UNIT, head, NULL);
    return root;
}
```

# 语法分析器parser.c文件

```
/* Decl: const int a = 1; | int a; | int a = 1; */
static past parseDecl(void) {
    if (g_look == Y_CONST) advance();
    int t = parseType();
    past vd = makeNode(VAR_DECL, NULL, NULL);
    vd->ivalue = t;
    if (g_look == Y_ID) {
        vd->svalue = yylval.id_name;
        advance();
    }
    if (g_look == Y_ASSIGN) {
        advance();
        vd->right = parseExp();
    }
    match(Y_SEMICOLON);
    return vd;
}
```

# 语法分析器parser.c文件

```
/* Block: { BlockItems } */
static past parseBlock(void) {
    match(Y_LBRACKET);
    past head = NULL;
    while (g_look != Y_RBRACKET && g_look != 0) {
        past item;
        if (g_look == Y_CONST || g_look == Y_INT || g_look == Y_FLOAT) {
            item = parseDecl();
        } else {
            item = parseStmt();
        }
        head = appendSibling(head, item);
    }
    match(Y_RBRACKET);
    past blk = makeNode(COMPOUND_STMT, head, NULL);
    return blk;
}
```

# 语法分析器parser.c文件

```
/* Stmt */
static past parseStmt(void) {
    if (g_look == Y_SEMICOLON) { advance();
       return makeNode(NULL_STMT, NULL, NULL);
    }
    if (g_look == Y_LBRACKET) { return parseBlock();  }
    if (g_look == Y_IF) {
        advance();
        match(Y_LPAR);
        past cond = parseLOrExp();
        match(Y_RPAR);
        past thenStmt = parseStmt();
        past elseStmt = NULL;
        if (g_look == Y_ELSE) { advance();
            elseStmt = parseStmt();
        }
        return makeIfNode(cond, thenStmt, elseStmt);
    }
```

# 语法分析器parser.c文件

```
if (g_look == Y_WHILE) {
    advance();
    match(Y_LPAR);
    pastcond = parseLOrExp();
    match(Y_RPAR);
    pastbody = parseStmt();
    returnmakeWhileNode(cond, body);
}
if (g_look == Y_BREAK) {
    advance(); match(Y_SEMICOLON);
    returnmakeNode(BREAK_STMT, NULL, NULL);
}
if (g_look == Y_CONTINUE) {
    advance(); match(Y_SEMICOLON);
    return makeNode(CONTINUE_STMT, NULL, NULL);
}
```

# 语法分析器parser.c文件

```
if (g_look == Y_RETURN) {
    advance();
    past ret = makeNode(RETURN_STMT, NULL, NULL);
    if (g_look != Y_SEMICOLON) {
        ret->left = parseExp();
    }
    match(Y_SEMICOLON);
    return ret;
}
```

# 语法分析器parser.c文件

```
/* 赋值语句: LVal = Exp ;  或 表达式语句: Exp ; */
if (g_look == Y_ID) {
    char* name = yylval.id_name;
    advance();
    if (g_look == Y_ASSIGN) {
        past lval = makeIdNode(DECL_REF_EXPR, name);
        lval->isLValue = 1;
        advance(); /* '=' */
        past rhs = parseExp();
        match(Y_SEMICOLON);
        /* 赋值语句用 BINARY_OPERATOR 节点表示，op = Y_ASSIGN */
        past assign = makeBinaryNode(BINARY_OPERATOR, Y_ASSIGN, lval, rhs);
        return assign;
```

# 语法分析器parser.c文件

```
        } else {
            /* 表达式语句：把 ID 当作 PrimaryExp 起头继续 */
            /* 简化：回退太麻烦，这里直接构造一个 DECL_REF_EXPR 并继续解析 */
            paste = makeIdNode(DECL_REF_EXPR, name);
            /* 继续解析后续运算符 */
            /* 为简化，这里只处理简单情况 */
            match(Y_SEMICOLON);
            return makeNode(UNEXPOSED_STMT, e, NULL);
        }
    }
    /* 其他表达式语句 */
    past e = parseExp();
    match(Y_SEMICOLON);
    return makeNode(UNEXPOSED_STMT, e, NULL);
}
```

# 语法分析器parser.c文件

```
/* ---- 表达式分层 ---- */
static past parseLOrExp(void) {
    past left = parseLAndExp();
    while (g_look == Y_OR) {
        int op = g_look; advance();
        past right = parseLAndExp();
        left = makeBinaryNode(BINARY_OPERATOR, op, left, right);
    }
    return left;
}
static past parseLAndExp(void) {
    past left = parseEqExp();
    while (g_look == Y_AND) {
        int op = g_look; advance(); past right = parseEqExp();
        left = makeBinaryNode(BINARY_OPERATOR, op, left, right);
    }
    return left;
}
```

# 语法分析器parser.c文件

```
static past parseEqExp(void) {
    past left = parseRelExp();
    while (g_look == Y_EQ || g_look == Y_NOTEQ) {
        int op = g_look; advance();
        past right = parseRelExp();
        left = makeBinaryNode(BINARY_OPERATOR, op, left, right);
    }
    return left;
}
static past parseRelExp(void) {
    past left = parseAddExp();
    while (g_look == Y_LESS || g_look == Y_LESSEQ ||
           g_look == Y_GREAT || g_look == Y_GREATEQ) {
        int op = g_look; advance();
        past right = parseAddExp();
        left = makeBinaryNode(BINARY_OPERATOR, op, left, right);
    }
    return left;
}
```

# 语法分析器parser.c文件

```
static past parseAddExp(void) {
    past left = parseMulExp();
    while (g_look == Y_ADD || g_look == Y_SUB) {
        intop = g_look; advance();
        past right = parseMulExp();
        left = makeBinaryNode(BINARY_OPERATOR, op, left, right);
    }
    return left;
}
static past parseMulExp(void) {
    past left = parseUnaryExp();
    while (g_look == Y_MUL || g_look == Y_DIV || g_look == Y_MODULO) {
        int op = g_look; advance();
        past right = parseUnaryExp();
        left = makeBinaryNode(BINARY_OPERATOR, op, left, right);
    }
    return left;
}
```

# 语法分析器parser.c文件

```
static past parseUnaryExp(void) {
    if (g_look == Y_ADD || g_look == Y_SUB || g_look == Y_NOT) {
        intop = g_look; advance();
        past child = parseUnaryExp();
        return makeUnaryNode(UNARY_OPERATOR, op, child);
    }
    return parsePrimaryExp();
}
static past parseExp(void) {
    return parseAddExp();
}
/* 对外入口 */
past parse(constchar* src) {
    lexer_init(src);
    advance();
    return parseCompUnit();
}
```

# 语法分析器parser.c文件

```
static past parsePrimaryExp(void) {
    if (g_look == Y_LPAR) {
        advance(); past e = parseExp();
        match(Y_RPAR); return e;
    }
    if (g_look == num_INT) {
        int v = yylval.int_value;
        advance(); return makeIntNode(v);
    }
    if (g_look == num_FLOAT) {
        float v = yylval.float_value;
        advance(); return makeFloatNode(v);
    }
    if (g_look == Y_ID) {
        char* name = yylval.id_name;
        advance(); return makeIdNode (DECL_REF_EXPR, name);
    }
    advance();
    return NULL;
}
```

# 语义分析semantic.h头文件

```
#ifndef COMPILER_SEMANTIC_H
#define COMPILER_SEMANTIC_H
#include"ast.h"
/* 符号表项 */
typedef struct _sym_entry {
    char*  name;
    int    dataType;     /* TY_VOID/TY_INT/TY_FLOAT */
    int    isGlobal;
    int    isConst;
    char*  irName;       /* LLVM IR 中对应的名字（@g 或 %p 等） */
    int    slot;         /* 局部变量在栈帧中的编号 */
    struct _sym_entry* next;
} sym_entry;
```

# 语义分析semantic.h头文件（续）

```
/* 作用域栈 */
typedef struct _scope {
    sym_entry*   head;
    struct _scope* parent;
} scope;
void semantic_init(void);
void semantic_finish(void);
/* 进入/离开作用域 */
void enter_scope(void);
void leave_scope(void);
/* 声明一个符号 */
sym_entry* declare_sym(const char* name, int type, int isGlobal, int isConst);
/* 查找符号（从内到外） */
sym_entry* lookup_sym(constchar* name);
/* 对整棵 AST 做语义分析（类型检查 + 符号表建立） */
void semantic_analyze(past root);
#endif
```

# 语义分析semantic.c文件

```c
#include "semantic.h"
#include <stdio.h>
#include <string.h>
static scope* g_global = NULL;
static scope* g_cur    = NULL;
static int g_tmp_slot = 0;
void semantic_init(void) {
    g_global = (scope*)calloc(1, sizeof(scope));
    g_cur = g_global;
    g_tmp_slot = 0;
}
void semantic_finish(void) {
    /* 实际释放略，教学实验省略 */
}
```

# 语义分析semantic.c文件（续）

```
void enter_scope(void) {
    scope* s = (scope*)calloc(1, sizeof(scope));
    s->parent = g_cur;
    g_cur = s;
}
void leave_scope(void) {
    /* 仅上移作用域指针，不释放符号表结点——IR 生成阶段仍需查询 */
    g_cur = g_cur->parent;
}
```

# 语义分析semantic.c文件（续）

```
sym_entry* declare_sym(const char* name, int type, int isGlobal, int isConst) {
    /* 同一作用域内重复声明检查 */
    sym_entry* p = g_cur->head;
    while (p) {
        if (strcmp(p->name, name) == 0) {
            fprintf(stderr, "semantic error: redeclaration of '%s'\n", name);
            return p;
        }
        p = p->next;
    }
```

# 语义分析semantic.c文件（续declare_sym函数）

```
    sym_entry* e = (sym_entry*)calloc(1, sizeof(sym_entry));
    e->name = strdup(name);
    e->dataType = type;
    e->isGlobal = isGlobal;
    e->isConst  = isConst;
    if (isGlobal) {
        e->irName = (char*)malloc(64);
        snprintf(e->irName, 64, "@%s", name);
    } else {
        e->slot = g_tmp_slot++;
        e->irName = (char*)malloc(64);
        snprintf(e->irName, 64, "%%.%d", e->slot);
    }
    e->next = g_cur->head;
    g_cur->head = e;
    return e;
}
```

# 语义分析semantic.c文件（续）

```
/* ---- 简单的类型推导 ---- */
static int expr_type(past node) {
    if (!node) return TY_VOID;
    switch (node->nodeType) {
        case INTEGER_LITERAL:  return TY_INT;
        case FLOATING_LITERAL: return TY_FLOAT;
        case DECL_REF_EXPR: {
            sym_entry* e = lookup_sym(node->svalue);
            return e ? e->dataType : TY_INT;
        }
        case BINARY_OPERATOR:
        case UNARY_OPERATOR: {
            int lt = expr_type(node->left);
            int rt = node->right ? expr_type(node->right) : TY_VOID;
            if (lt == TY_FLOAT || rt == TY_FLOAT) return TY_FLOAT;
            if (lt == TY_INT || rt == TY_INT)    return TY_INT;
            return TY_VOID;
        }
        default:
            return node->dataType;
    }
}
```

# 语义分析semantic.c文件（续）

```
void semantic_analyze(past root) {
    if (!root) return;
    /* 教学实验：只做符号登记与类型回填，不做完整错误恢复 */
    past it = root->left; /* CompUnit 的兄弟链 */
    while (it) {
        if (it->nodeType == VAR_DECL) {
            int ty = (it->ivalue == Y_FLOAT) ? TY_FLOAT : TY_INT;
            declare_sym(it->svalue, ty, 1, 0);
            it->dataType = ty;
        } else if (it->nodeType == FUNCTION_DECL) {
            int ty = (it->ivalue == Y_FLOAT) ? TY_FLOAT :
                     (it->ivalue == Y_VOID) ? TY_VOID : TY_INT;
            declare_sym(it->svalue, ty, 1, 1);
            it->dataType = ty;
            /* 进入函数体作用域 */
            enter_scope();
```

# 语义分析semantic.c文件（续semantic_analyze）

```
            /* 函数体内语句做类型回填 */
            past blk = it->left;
            past bi = blk ? blk->left : NULL;
            while (bi) {
                if (bi->nodeType == VAR_DECL) {
                    int bty = (bi->ivalue == Y_FLOAT) ? TY_FLOAT : TY_INT;
                    declare_sym(bi->svalue, bty, 0, 0);
                    bi->dataType = bty;
                } else if (bi->nodeType == BINARY_OPERATOR && bi->ivalue == Y_ASSIGN) {
                    /* 赋值语句：回填类型 */
                    int rt = expr_type(bi->right);
                    bi->dataType = rt;
                } else if (bi->nodeType == IF_STMT || bi->nodeType == WHILE_STMT) {
                    /* 条件语句的条件类型 */
                    if (bi->if_cond) bi->if_cond->dataType = expr_type(bi->if_cond);
                }
                bi = bi->next;
            }
            leave_scope();
        }
        it = it->next;
    }
}
```

# 语义分析semantic.c文件（续semantic_analyze）

```
            /* 函数体内语句做类型回填 */
            past blk = it->left;
            past bi = blk ? blk->left : NULL;
            while (bi) {
                if (bi->nodeType == VAR_DECL) {
                    int bty = (bi->ivalue == Y_FLOAT) ? TY_FLOAT : TY_INT;
                    declare_sym(bi->svalue, bty, 0, 0);
                    bi->dataType = bty;
                } else if (bi->nodeType == BINARY_OPERATOR && bi->ivalue == Y_ASSIGN) {
                    /* 赋值语句：回填类型 */
                    int rt = expr_type(bi->right);
                    bi->dataType = rt;
                } else if (bi->nodeType == IF_STMT || bi->nodeType == WHILE_STMT) {
                    /* 条件语句的条件类型 */
                    if (bi->if_cond) bi->if_cond->dataType = expr_type(bi->if_cond);
                }
                bi = bi->next;
            }
            leave_scope();
        }
        it = it->next;
    }
}
```

# 代码生成 ir_gen.h头文件

```
#ifndefCOMPILER_IR_GEN_H
#defineCOMPILER_IR_GEN_H
#include"ast.h"
/*
 * LLVM IR 中间代码生成模块
 *   - result 为可动态增长的 char 缓冲区，IR 指令以"instr\n" 的形式追加到末尾，每行仅一条指令。
 *   - 返回值为该表达式结点求值结果在 IR 中的值名（如 "%t3"），
 *     供上层调用方继续引用；语句类函数无返回值。*/
/* 初始化 IR 生成器（清零计数器、清空缓冲区） */
void        ir_gen_init(void);
/* 从 AST 根出发生成完整模块级 IR，输出到 out_buf（自动追加换行） */
void        ir_gen_module(past root, char* out_buf, int out_size);
const char* genArithmeticExpr(past node, char* result);
const char* genLogicExpr     (past node, char* result);
const char* genAssignStmt    (past node, char* result);
void        genIfStmt        (past node, char* result);
void        genWhileStmt     (past node, char* result);
/* ---- 其它按需设计的辅助函数 ---- */
const char* genExpr          (past node, char* result);   /* 表达式总入口 */
const char* genLValueAddr    (past node, char* result);   /* 取左值地址 */
void        genStmt          (past node, char* result);   /* 语句总入口 */
void        genBlock         (past node, char* result);   /* 复合语句 */
#endif
```

# 代码生成 ir_gen.c文件

```c
#include"ir_gen.h"
#include"semantic.h"
#include<stdio.h>
#include<string.h>
#include<stdarg.h>
/* ====================== 全局 IR 生成状态 ====================== */
static int g_temp_no = 0;
static int g_label_no = 0;
static int g_func_ret_type = TY_INT;
/* 全局字符串池：保证 new_temp / new_label / 字面量操作数返回的指针长期有效 */
static char g_temp_names[2048][64];
static char g_label_names[512][64];
static char g_val_buf[2048][64];
static int  g_val_idx = 0;
```

# 代码生成 ir_gen.c文件(续)

```c
/* break / continue 标签栈（while 嵌套） */
#define MAX_LOOP_DEPTH 64
static char g_break_labels[MAX_LOOP_DEPTH][64];
static char g_cont_labels[MAX_LOOP_DEPTH][64];
static int  g_loop_depth = 0;
/* 类型字符串 */
static const char* ty_str(int ty) {
    if (ty == TY_FLOAT) return "float";
    if (ty == TY_INT)   return "i32";
    return "void";
}
/* 分配新临时寄存器名（返回全局池中永久字符串） */
static const char* new_temp(void) {
    snprintf(g_temp_names[g_temp_no], 64, "%%t%d", g_temp_no);
    return g_temp_names[g_temp_no++];
}
```

# 代码生成 ir_gen.c文件(续)

```c
/* 分配新基本块标签名 */
static const char* new_label(void) {
    snprintf(g_label_names[g_label_no], 64, "%%L%d", g_label_no);
    return g_label_names[g_label_no++];
}
/* 向 result 追加一行指令（带换行） */
static void emit(char* result, const char* fmt, ...) {
    va_list ap;
    va_start(ap, fmt);
    char line[512];
    vsnprintf(line, sizeof(line), fmt, ap);
    va_end(ap);
    strcat(result, line);
    strcat(result, "\n");
}
```

# 代码生成 ir_gen.c文件(续)

```c
/* ====================== 表达式取值辅助 ====================== */
/* 取变量在 IR 中的当前值（load） */
static const char* load_var(const char* irName, int ty, char* result) {
    const char* t = new_temp();
    emit(result, "%s = load %s, %s* %s", t, ty_str(ty), ty_str(ty), irName);
    return t;
}
/* 整数字面量 → 从全局池取出操作数字符串 */
static const char* int_operand(int val, int ty) {
    char* buf = g_val_buf[g_val_idx++];
    if (ty == TY_FLOAT) snprintf(buf, 64, "%d.0", val);
    else                snprintf(buf, 64, "%d", val);
    return buf;
}
```

# 代码生成 ir_gen.c文件(续)

```c
/* 浮点字面量 → 从全局池取出操作数字符串 */
static const char* float_operand(float val, int ty) {
    char* buf = g_val_buf[g_val_idx++];
    if (ty == TY_INT)   snprintf(buf, 64, "%d", (int)val);
    else                snprintf(buf, 64, "%f", val);
    return buf;
}
```

# 代码生成 ir_gen.c文件

```
/* ====================== 算术表达式 genArithmeticExpr ====================== */
const char* genArithmeticExpr(past node, char* result) {
    if (!node) return "undef";
    if (node->nodeType == INTEGER_LITERAL)
        return int_operand(node->ivalue, node->dataType);
    if (node->nodeType == FLOATING_LITERAL)
        return float_operand(node->fvalue, node->dataType);
    if (node->nodeType == DECL_REF_EXPR) {
        sym_entry* e = lookup_sym(node->svalue);
        if (!e) return "undef";
        node->dataType = e->dataType;
        return load_var(e->irName, e->dataType, result);
    }
    if (node->nodeType == PAREN_EXPR)
        return genArithmeticExpr(node->left, result);
```

# 代码生成 ir_gen.c文件(续genArithmeticExp)

```
/* 一元运算 */
    if (node->nodeType == UNARY_OPERATOR) {
        const char* v = genArithmeticExpr(node->left, result);
        int ty = node->left->dataType;
        node->dataType = ty;
        if (node->ivalue == Y_SUB) {
            const char* t = new_temp();
            if (ty == TY_FLOAT)
                emit(result, "%s = fneg float %s", t, v);
            else
                emit(result, "%s = sub i32 0, %s", t, v);
            return t;
        }
        return v;
    }
```

# 代码生成 ir_gen.c文件(续genArithmeticExp)

```
/* 二元运算 */
    if (node->nodeType == BINARY_OPERATOR) {
        int op = node->ivalue;
        if (op == Y_ASSIGN)
            return genAssignStmt(node, result);
        if (op == Y_LESS || op == Y_LESSEQ || op == Y_GREAT ||
            op == Y_GREATEQ || op == Y_EQ   || op == Y_NOTEQ ||
            op == Y_AND    || op == Y_OR    || op == Y_NOT)
            return genLogicExpr(node, result);
        const char* l = genArithmeticExpr(node->left, result);
        const char* r = genArithmeticExpr(node->right, result);
        int ty = node->left->dataType;
        if (node->right->dataType == TY_FLOAT) ty = TY_FLOAT;
        node->dataType = ty;
        const char* t = new_temp();
        const char* ts = ty_str(ty);
```

# 代码生成 ir_gen.c文件(续genArithmeticExp)

```
switch (op) {
    case Y_ADD:
        if (ty == TY_FLOAT) emit(result, "%s = fadd float %s, %s", t, l, r);
        else                emit(result, "%s = add i32 %s, %s", t, l, r);
        break;
    case Y_SUB:
        if (ty == TY_FLOAT) emit(result, "%s = fsub float %s, %s", t, l, r);
        else                emit(result, "%s = sub i32 %s, %s", t, l, r);
        break;
    case Y_MUL:
        if (ty == TY_FLOAT) emit(result, "%s = fmul float %s, %s", t, l, r);
        else                emit(result, "%s = mul i32 %s, %s", t, l, r);
        break;
    case Y_DIV:
        if (ty == TY_FLOAT) emit(result, "%s = fdiv float %s, %s", t, l, r);
        else                emit(result, "%s = sdiv i32 %s, %s", t, l, r);
        break;
```

# 代码生成 ir_gen.c文件(续genArithmeticExp)

```
            case Y_MODULO:
                if (ty == TY_FLOAT) emit(result, "%s = frem float %s, %s", t, l, r);
                else                emit(result, "%s = srem i32 %s, %s", t, l, r);
                break;
            default:
                emit(result, "%s = add %s %s, %s ; unknown op", t, ts, l, r);
                break;
        }
        return t;
    }
    return "undef";
}
```

# 代码生成 ir_gen.c文件(续)

```
/* ====================== 逻辑表达式 genLogicExpr ====================== */
const char* genLogicExpr(past node, char* result) {
    if (!node) return "false";
    /* 逻辑非 !x */
    if (node->nodeType == UNARY_OPERATOR && node->ivalue == Y_NOT) {
        const char* v = genLogicExpr(node->left, result);
        const char* t = new_temp();
        emit(result, "%s = xor i1 %s, true", t, v);
        node->dataType = TY_INT;
        return t;
    }
    if (node->nodeType != BINARY_OPERATOR) {
        const char* v = genArithmeticExpr(node, result);
        int ty = node->dataType;
        const char* t = new_temp();
        if (ty == TY_FLOAT)
            emit(result, "%s = fcmp one float %s, 0.0", t, v);
        else
            emit(result, "%s = icmp ne i32 %s, 0", t, v);
        return t;
    }
    int op = node->ivalue;
```

# 代码生成 ir_gen.c文件(续genLogicExpr)

```
    /* ---- 关系运算 ---- */
    if (op == Y_LESS || op == Y_LESSEQ || op == Y_GREAT ||
        op == Y_GREATEQ || op == Y_EQ   || op == Y_NOTEQ) {
        const char* l = genArithmeticExpr(node->left, result);
        const char* r = genArithmeticExpr(node->right, result);
        int ty = node->left->dataType;
        if (node->right->dataType == TY_FLOAT) ty = TY_FLOAT;
        const char* t = new_temp();
        const char* icmp_op = "";
        const char* fcmp_op = "";
        switch (op) {
            caseY_LESS:    icmp_op = "slt"; fcmp_op = "olt"; break;
            caseY_LESSEQ:  icmp_op = "sle"; fcmp_op = "ole"; break;
            caseY_GREAT:   icmp_op = "sgt"; fcmp_op = "ogt"; break;
            caseY_GREATEQ: icmp_op = "sge"; fcmp_op = "oge"; break;
            caseY_EQ:      icmp_op = "eq";  fcmp_op = "oeq"; break;
            caseY_NOTEQ:   icmp_op = "ne";  fcmp_op = "one"; break;
        }
        if (ty == TY_FLOAT)
            emit(result, "%s = fcmp %s float %s, %s", t, fcmp_op, l, r);
        else
            emit(result, "%s = icmp %s i32 %s, %s", t, icmp_op, l, r);
        node->dataType = TY_INT;
        return t;
    }
```

# 代码生成 ir_gen.c文件(续genLogicExpr)

```
/* ---- 逻辑与 &&（短路求值） ---- */
    if (op == Y_AND) {
        const char* lbl_false = new_label();
        const char* lbl_end   = new_label();
        const char* lbl_true  = new_label();
        const char* l = genLogicExpr(node->left, result);
        emit(result, "br i1 %s, label %s, label %s", l, lbl_true, lbl_false);
        emit(result, "%s:", lbl_true);
        const char* r = genLogicExpr(node->right, result);
        emit(result, "br label %s", lbl_end);
        emit(result, "%s:", lbl_false);
        emit(result, "br label %s", lbl_end);
        emit(result, "%s:", lbl_end);
        const char* final_t = new_temp();
        emit(result, "%s = phi i1 [ false, %s ], [ %s, %s ]",
             final_t, lbl_false, r, lbl_true);
        node->dataType = TY_INT;
        return final_t;
    }
```

# 代码生成 ir_gen.c文件(续genLogicExpr)

```
    /* ---- 逻辑或 ||（短路求值） ---- */
     if (op == Y_OR) {
         const char* lbl_true  = new_label();
         const char* lbl_end   = new_label();
         const char* lbl_false = new_label();
         const char* l = genLogicExpr(node->left, result);
         emit(result, "br i1 %s, label %s, label %s", l, lbl_true, lbl_false);
         emit(result, "%s:", lbl_true);
         emit(result, "br label %s", lbl_end);
         emit(result, "%s:", lbl_false);
         const char* r = genLogicExpr(node->right, result);
         emit(result, "br label %s", lbl_end);
         emit(result, "%s:", lbl_end);
         const char* final_t = new_temp();
         emit(result, "%s = phi i1 [ true, %s ], [ %s, %s ]",
              final_t, lbl_true, r, lbl_false);
         node->dataType = TY_INT;
         return final_t;
     }
     return "false";
 }
```

# 代码生成 ir_gen.c文件(续)

```
/* ====================== 赋值语句 genAssignStmt ====================== */
const char* genAssignStmt(past node, char* result) {
    if (!node || node->ivalue != Y_ASSIGN) return NULL;
    past lhs = node->left;
    past rhs = node->right;
    const char* val;
    if (rhs->nodeType == BINARY_OPERATOR &&
        (rhs->ivalue == Y_AND || rhs->ivalue == Y_OR)) {
        val = genLogicExpr(rhs, result);
    } else {
        val = genArithmeticExpr(rhs, result);
    }
    sym_entry* e = lookup_sym(lhs->svalue);
    if (!e) return NULL;
    int ty = e->dataType;
    emit(result, "store %s %s, %s* %s",
         ty_str(ty), val, ty_str(ty), e->irName);
    node->dataType = ty;
    return val;
}
```

# 代码生成 ir_gen.c文件(续)

```
/* ====================== if 语句 genIfStmt ====================== */
void genIfStmt(past node, char* result) {
    if (!node) return;
    const char* cond = genLogicExpr(node->if_cond, result);
    const char* lbl_then = new_label();
    const char* lbl_else = new_label();
    const char* lbl_end  = new_label();
    emit(result, "br i1 %s, label %s, label %s", cond, lbl_then, lbl_else);
    emit(result, "%s:", lbl_then);
    if (node->left) genStmt(node->left, result);
    emit(result, "br label %s", lbl_end);
    emit(result, "%s:", lbl_else);
    if (node->right) genStmt(node->right, result);
    emit(result, "br label %s", lbl_end);
    emit(result, "%s:", lbl_end);
}
```

# 代码生成 ir_gen.c文件(续)

```
/* ====================== while 语句 genWhileStmt ====================== */
void enWhileStmt(past node, char* result) {
    if (!node) return;
    constchar* lbl_header = new_label();
    constchar* lbl_body   = new_label();
    constchar* lbl_exit   = new_label();
    snprintf(g_break_labels[g_loop_depth], 64, "%s", lbl_exit);
    snprintf(g_cont_labels[g_loop_depth],  64, "%s", lbl_header);
    g_loop_depth++;
    emit(result, "br label %s", lbl_header);
    emit(result, "%s:", lbl_header);
    constchar* cond = genLogicExpr(node->if_cond, result);
    emit(result, "br i1 %s, label %s, label %s", cond, lbl_body, lbl_exit);
    emit(result, "%s:", lbl_body);
    if (node->left) genStmt(node->left, result);
    emit(result, "br label %s", lbl_header);
    emit(result, "%s:", lbl_exit);
    g_loop_depth--;
}
```

# 代码生成 ir_gen.c文件(续)

```
/* ====================== 语句总入口 genStmt ====================== */
void genStmt(past node, char* result) {
    if (!node) return;
    switch (node->nodeType) {
        case COMPOUND_STMT:
            genBlock(node, result);
            break;
        case IF_STMT:
            genIfStmt(node, result);
            break;
        case WHILE_STMT:
            genWhileStmt(node, result);
            break;
        case BREAK_STMT:
            if (g_loop_depth > 0)
                emit(result, "br label %s", g_break_labels[g_loop_depth - 1]);
            break;
```

# 代码生成 ir_gen.c文件(续genStmt)

```
        case CONTINUE_STMT:
            if (g_loop_depth > 0)
                emit(result, "br label %s", g_cont_labels[g_loop_depth - 1]);
            break;
        case RETURN_STMT: {
            if (node->left) {
                const char* v = genArithmeticExpr(node->left, result);
                emit(result, "ret %s %s", ty_str(g_func_ret_type), v);
            } else {
                emit(result, "ret %s", ty_str(g_func_ret_type));
            }
            break;
        }
```

# 代码生成 ir_gen.c文件(续genStmt)

```
        case BINARY_OPERATOR:
            if (node->ivalue == Y_ASSIGN)
                genAssignStmt(node, result);
            else
                genExpr(node, result);
            break;
        case NULL_STMT:
            break;
        case UNEXPOSED_STMT:
            genExpr(node->left, result);
            break;
        default:
            genExpr(node, result);
            break;
    }
}
```

# 代码生成 ir_gen.c文件(续)

```
/* 复合语句 { ... } */
void genBlock(past node, char* result) {
    past it = node->left;
    while (it) {
        genStmt(it, result);
        it = it->next;
    }
}
/* 表达式总入口 */
constchar* genExpr(past node, char* result) {
    if (!node) return "undef";
    if (node->nodeType == BINARY_OPERATOR &&
        (node->ivalue == Y_AND || node->ivalue == Y_OR)) {
        return genLogicExpr(node, result);
    }
    if (node->nodeType == BINARY_OPERATOR &&
        (node->ivalue == Y_LESS || node->ivalue == Y_LESSEQ ||
         node->ivalue == Y_GREAT || node->ivalue == Y_GREATEQ ||
         node->ivalue == Y_EQ   || node->ivalue == Y_NOTEQ)) {
        return genLogicExpr(node, result);
    }
    return genArithmeticExpr(node, result);
}
```

# 代码生成 ir_gen.c文件(续)

```c
/* ======================== 模块级入口 ======================== */
void ir_gen_init(void) {
    g_temp_no = 0;
    g_label_no = 0;
    g_loop_depth = 0;
    g_val_idx = 0;
}
```

# 代码生成 ir_gen.c文件(续ir_gen_modul)

```
void ir_gen_module(past root, char* out_buf, int out_size) {
 memset(out_buf, 0, out_size);
 strcat(out_buf, "; 模块由 SysY 编译器自动生成\n");
 strcat(out_buf, "target datalayout = \"e-m:e-p:32:32\"\n");
 strcat(out_buf, "target triple = \"i686-pc-linux\"\n\n");
 pastit = root ? root->left : NULL;
 while (it) {
     if (it->nodeType == VAR_DECL) {
         int ty = it->ivalue;
         constchar* ts = (ty == Y_FLOAT) ? "float" : "i32";
         if (it->right) {
             char init_expr[8192];
             memset(init_expr, 0, sizeof(init_expr));
             constchar* v = genArithmeticExpr(it->right, init_expr);
             emit(out_buf, "@%s = global %s %s", it->svalue, ts, v);
         } else {
             emit(out_buf, "@%s = global %s 0", it->svalue, ts);
         }
```

# 代码生成 ir_gen.c文件(续)

```
void ir_gen_module(past root, char* out_buf, int out_size) {
 memset(out_buf, 0, out_size);
 strcat(out_buf, "; 模块由 SysY 编译器自动生成\n");
 strcat(out_buf, "target datalayout = \"e-m:e-p:32:32\"\n");
 strcat(out_buf, "target triple = \"i686-pc-linux\"\n\n");
 pastit = root ? root->left : NULL;
 while (it) {
     if (it->nodeType == VAR_DECL) {
         int ty = it->ivalue;
         constchar* ts = (ty == Y_FLOAT) ? "float" : "i32";
         if (it->right) {
             char init_expr[8192];
             memset(init_expr, 0, sizeof(init_expr));
             constchar* v = genArithmeticExpr(it->right, init_expr);
             emit(out_buf, "@%s = global %s %s", it->svalue, ts, v);
         } else {
             emit(out_buf, "@%s = global %s 0", it->svalue, ts);
         }
```

# 代码生成 ir_gen.c文件(续ir_gen_modul)

```
} else if (it->nodeType == FUNCTION_DECL) {
        g_func_ret_type = (it->ivalue == Y_FLOAT) ? TY_FLOAT :
                          (it->ivalue == Y_VOID) ? TY_VOID : TY_INT;
        const char* ret_ts = ty_str(g_func_ret_type);
        emit(out_buf, "define %s @%s() {", ret_ts, it->svalue);
        emit(out_buf, "entry:");
        /* 进入函数作用域，重新登记局部变量（供 IR 生成 lookup） */
        enter_scope();
        /* 为函数体内所有局部变量生成 alloca，并同步登记到符号表 */
        past blk = it->left;
        past bi = blk ? blk->left : NULL;
```

# 代码生成 ir_gen.c文件(续ir_gen_modul)

```
while (bi) {
    if (bi->nodeType == VAR_DECL) {
        intbty = (bi->ivalue == Y_FLOAT) ? TY_FLOAT : TY_INT;
        sym_entry* e = declare_sym(bi->svalue, bty, 0, 0);
        if (e) {
            emit(out_buf, "%s = alloca %s", e->irName, ty_str(e->dataType));
            if (bi->right) {
                charinit_buf[4096];
                memset(init_buf, 0, sizeof(init_buf));
                constchar* v = genArithmeticExpr(bi->right, init_buf);
                strcat(out_buf, init_buf);
                emit(out_buf, "store %s %s, %s* %s",
                     ty_str(e->dataType), v, ty_str(e->dataType), e->irName);
            }
        }
    }
    bi = bi->next;
}
```

# 代码生成 ir_gen.c文件(续ir_gen_modul)

```
            /* 函数体语句（跳过已处理的 VAR_DECL） */
            bi = blk ? blk->left : NULL;
            while (bi) {
                if (bi->nodeType != VAR_DECL) {
                    genStmt(bi, out_buf);
                }
                bi = bi->next;
            }
            leave_scope();
            emit(out_buf, "}");
            strcat(out_buf, "\n");
        }
        it = it->next;
    }
}
```

# 测试程序gencode.c文件

```c
/*
 * gencode.c – 编译器前端测试入口
 *
 * 用法:
 *   ./gencode example.sysy          （从文件读源码）
 *   ./gencode                       （从 stdin 读源码）
 *
 * 流程: 词法分析 → 语法分析（递归下降建 AST）→ 语义分析（符号表）→ LLVM IR 生成
 */
#include "ast.h"
#include "lexer.h"
#include "parser.h"
#include "semantic.h"
#include "ir_gen.h"
#include <stdio.h>
#include <stdlib.h>
```

# 测试程序gencode.c文件

```
int main(int argc, char** argv) {
    char* src = NULL;
    if (argc >= 2) {
        FILE* fp = fopen(argv[1], "rb");
        if (!fp) { perror("fopen"); return1; }
        fseek(fp, 0, SEEK_END);
        long n = ftell(fp);
        fseek(fp, 0, SEEK_SET);
        src = (char*)malloc(n + 1);
        fread(src, 1, n, fp);
        src[n] = '\0';
        fclose(fp);
```

# 测试程序gencode.c文件

```
    } else {
        /* 从 stdin 读 */
        size_t cap = 4096, len = 0;
        src = (char*)malloc(cap);
        int c;
        while ((c = getchar()) != EOF) {
            if (len + 1 >= cap) { cap *= 2; src = (char*)realloc(src, cap); }
            src[len++] = (char)c;
        }
        src[len] = '\0';
    }
```

# 测试程序gencode.c文件

```
/* 1) 语法分析（内部已含词法） */
    past root = parse(src);
    /* 2) 语义分析 */
    semantic_init();
    semantic_analyze(root);
    /* 3) LLVM IR 生成 */
    ir_gen_init();
    char* ir = (char*)malloc(1024 * 1024);    /* 1MB 缓冲 */
    ir_gen_module(root, ir, 1024 * 1024);
    printf("%s", ir);
    freeAst(root);
    free(src);
    free(ir);
    return 0;
}
```

# 编译运行

**该程序在D:\myDemo\gencode目录下，运行编译：**

**gcc -g -Wall -I. -o gencode.exe ast.c lexer.c parser.c semantic.c ir_gen.c gencode.c**

**执行：gencode example.sysy**

```
D:\myDemo\gencode>gcc -g -Wall -I. -o gencode.exe ast.c lexer.c parser.c semantic.c ir_gen.c gencode.c

D:\myDemo\gencode>gencode example.sysy
```

```
/* 示例 SysY 程序：example.sysy */
int sum = 0;
int i = 1;

int main() {
    int n = 10;
    while (i <= n) {
        sum = sum + i;
        i = i + 1;
    }
    if (sum > 50) {
        return sum;
    } else {
        return 0;
    }
}
```



```
; 模块由 SysY 编译器自动生成
target datalayout = "e-m:e-p:32:32"
target triple = "i686-pc-linux"

@sum = global i32 0
@i = global i32 1
define i32 @main() {
entry:
%.1 = alloca i32
store i32 10, i32* %.1
br label %L0
%L0:
%t0 = load i32, i32* @i
%t1 = load i32, i32* %.1
%t2 = icmp sle i32 %t0, %t1
br i1 %t2, label %L1, label %L2
%L1:
%t3 = load i32, i32* @sum
%t4 = load i32, i32* @i
%t5 = add i32 %t3, %t4
store i32 %t5, i32* @sum
%t6 = load i32, i32* @i
%t7 = add i32 %t6, 1
store i32 %t7, i32* @i
br label %L0
%L2:
%t8 = load i32, i32* @sum
%t9 = icmp sgt i32 %t8, 50
br i1 %t9, label %L3, label %L4
%L3:
%t10 = load i32, i32* @sum
ret i32 %t10
br label %L5
%L4:
ret i32 0
br label %L5
%L5:
}
```